TÉCNICAS PARA SUSTENTAR LEITURA E ESCRITA

Base: mapeamento "Pressão de leitura e escrita por fluxo" (cenário inicial de 6 mil freelancers e 200 empresas, e cenário 10x), modelo físico em PostgreSQL 16, topologia geral (balanceador, 2 ou mais servidores de aplicação stateless, worker, PostgreSQL 16 com fila de tarefas, backup diário) e requisitos RNF08, RNF13, RNF15, RNF26, RNF30.

1. PONTO DE PARTIDA

O mapeamento mostrou os fluxos críticos nesta ordem:

Posição	Fluxo	Pressão	Volume no pico (10x)
1	Notificações em massa	Escrita em rajada (1 vaga = 1.200 avisos em até 5 s)	133 escritas/s em média, rajadas de milhares
2	Busca de vagas	Leitura constante, prazo de 3 s	40 leituras/s
3	Candidatura/contratação	Escrita transacional com concorrência	10 leituras/s e 8 escritas/s
4	Chat	Leitura alta só se usar polling	11 leituras/s (90 com polling)
5	Pagamento	Baixo volume, consistência obrigatória	1,5 operação/s

Esses números cabem com folga em um único PostgreSQL 16 bem indexado. Por isso a regra geral das decisões é: resolver primeiro dentro do PostgreSQL (índices, transações, fila em tabela, particionamento) e só acrescentar peça nova de infraestrutura quando ela resolve algo que o banco não resolve bem sozinho.

FREELAS

2. TÉCNICAS ESCOLHIDAS

2.1 Índices (PostgreSQL B-tree)

Escolha: índices compostos na ordem das consultas mais frequentes, e índices parciais quando a consulta sempre filtra o mesmo status.

Índice	Fluxo atendido	Consulta que acelera
vaga (funcao_id, inicio) WHERE status = 'PUBLICADA'	Busca de vagas	Listar vagas abertas de uma função por data
notificacao (usuario_id, criada_em DESC) WHERE lida = FALSE	Notificações	Contador e lista de não lidas ao abrir o app
mensagem (conversa_id, enviada_em)	Chat	Carregar o histórico de uma conversa
candidatura (vaga_id, status)	Candidatura	Empresa lista candidatos de uma vaga
contratacao (status, confirmada_em)	Contratação, pagamento	Tarefas agendadas de check-in e liberação

Índice	Fluxo atendido	Consulta que acelera
freelancer_funcao (funcao_id, freelancer_id)	Notificações, substituição	Achar elegíveis por função

Justificativa:

- O índice parcial de vaga só guarda vagas PUBLICADA. Vagas concluídas e canceladas, que serão a maioria com o tempo, ficam de fora, então o índice continua pequeno e rápido. Atende o RNF08 (3 s).
- O índice parcial de notificacao só guarda as não lidas. A tabela cresce cerca de 5 milhões de linhas por mês, mas o contador do app só olha as não lidas de um usuário.
- Hoje notificacao e mensagem só têm a chave primária. Sem esses índices, as consultas mais frequentes do app varreriam a tabela inteira.
- Não foram criados índices em todas as colunas: cada índice a mais deixa a escrita mais lenta, e notificacao é a tabela que mais recebe escrita.

2.2 Fila de tarefas (tabela no PostgreSQL)

Escolha: a fila de tarefas prevista na topologia fica numa tabela do próprio PostgreSQL (fila_tarefa), lida pelo worker com SELECT ... FOR UPDATE SKIP LOCKED. O worker é acordado na hora com LISTEN/NOTIFY.

Tarefas que passam pela fila:

- Avisar freelancers de vaga nova (RNF15, RNF22) e de substituição.
- Enviar push e e-mail pelo Serviço de Notificações.
- Processar o webhook do Gateway de Pagamento (o backend responde 200 e enfileira).
- Liberar pagamento quando o prazo de tolerância vence.
- Gravar log_auditoria.

Justificativa:

- A tarefa é gravada na mesma transação que a gerou (padrão outbox). Se a vaga foi publicada, a tarefa de aviso existe; se a transação falhou, nenhuma das duas existe. Com um serviço de fila separado, seria possível publicar a vaga e perder o aviso.
- SKIP LOCKED deixa vários workers pegarem tarefas diferentes sem disputar a mesma linha, então dá para subir mais workers no cenário 10x.
- Volume: mesmo no cenário 10x são poucas centenas de tarefas por segundo, o que uma tabela indexada no PostgreSQL 16 suporta. Um serviço como

RabbitMQ ou Kafka seria mais uma peça para instalar, monitorar e fazer backup sem necessidade para esse volume.

- Falha e repetição: cada tarefa guarda o número de tentativas e a próxima tentativa. Se o Serviço de Notificações estiver fora, a tarefa volta para a fila em vez de se perder.

2.3 Escrita em lote para as notificações em massa

Escolha: o aviso de vaga nova é gravado com um único comando INSERT ... SELECT, que busca os elegíveis e grava todos os avisos de uma vez, dentro da tarefa do worker. O push sai em lotes de até 500 aparelhos por chamada ao Serviço de Notificações.

Justificativa:

- 1.200 comandos INSERT separados significam 1.200 idas ao banco. Um comando que grava 1.200 linhas é uma ida só e uma única gravação no log do banco.
- Como roda no worker, a empresa recebe a confirmação de "vaga publicada" na hora e não espera os avisos saírem.
- O prazo de 5 s (RNF13, RNF15) passa a depender só do worker, que é acordado imediatamente pelo LISTEN/NOTIFY.

2.4 Particionamento e retenção

Escolha: notificacao e log_auditoria particionadas por mês (particionamento declarativo do PostgreSQL 16). Avisos com mais de 90 dias são apagados removendo a partição inteira. O log de auditoria guarda as partições pelo prazo que o grupo definir para o RNF04 e depois as move para o armazenamento de arquivos.

Justificativa:

- São as duas tabelas que mais crescem e nunca são editadas depois de lidas.
- Apagar uma partição é instantâneo, e um DELETE de milhões de linhas trava a tabela e deixa espaço morto.
- As consultas do app sempre filtram pelos últimos dias, então o PostgreSQL só lê a partição do mês atual.
- Consequência no modelo: a chave primária dessas tabelas precisa incluir a coluna da data, (id, criada_em) em notificacao e (id, registrado_em) em log_auditoria, para o particionamento funcionar. Nenhuma outra tabela aponta para essas duas, então a troca não quebra chaves estrangeiras.

2.5 Réplica de leitura

Escolha: uma réplica do PostgreSQL 16 por replicação em streaming assíncrona. A réplica atende só as leituras que toleram até alguns segundos de atraso:

Vai para a réplica	**Fica no banco principal**
Listagem e detalhe de vagas	Qualquer leitura que confere uma regra antes de gravar (candidatura duplicada, vaga lotada, idempotência do pagamento)
Perfis, lista de candidatos, avaliações	Tela logo após o próprio usuário gravar algo (minhas candidaturas após candidatar, conversa após enviar mensagem)
Histórico antigo do chat	Carteira, extrato e pagamentos
Relatórios e consultas de auditoria	Substituição ("primeiro que aceitar")

Justificativa:

- No cenário inicial o banco principal aguenta tudo sozinho. A réplica entra desde o lançamento principalmente para disponibilidade: se o servidor principal cair, a réplica é promovida em minutos, o que o backup diário sozinho não resolve (perderia até um dia de dados).
- No cenário 10x a busca de vagas é a maior carga de leitura. Mandá-la para a réplica deixa o principal livre para as escritas das notificações e da contratação.
- O atraso normal da réplica é menor que 1 s, dentro dos 5 s do RNF15 para a vaga nova aparecer.
- Leituras de dinheiro e de regra de negócio ficam no principal porque um dado atrasado ali gera erro real (saldo errado, duas contratações para a mesma vaga).

2.6 Cache

Escolha: um cache em memória compartilhado (Redis gerenciado pelo provedor de nuvem) usado em três casos. É uma peça nova na topologia, ao lado dos servidores de aplicação.

O que fica no cache	**Validade**	**Fluxo**
Resultado do Serviço de Mapas (CEP para região e distância)	30 dias	Busca, substituição
Primeira página da busca por função e cidade	60 s, apagada quando uma vaga dessa função é publicada ou preenchida	Busca de vagas
Contadores do limite de requisições (RNF30)	1 min	Todos

Dados que quase não mudam (lista de funções, planos e preços) ficam em memória em cada servidor de aplicação, sem Redis.

O que não vai para o cache: carteira, pagamento, status de candidatura e contratação, e mensagens. São dados que mudam a cada ação e precisam estar sempre certos.

Justificativa:

- Mapas: sem coordenadas no banco (decisão de 07/10), a distância vem do Serviço de Mapas, que tem até 2 s de resposta (RNF26) e cobra por chamada. CEPs se repetem muito numa mesma cidade, então guardar a resposta corta a maior parte das chamadas e mantém a busca dentro dos 3 s do RNF08.
- Busca: muitos freelancers da mesma cidade e função abrem a mesma primeira página no pico. Apagar o cache quando uma vaga é publicada garante que a vaga nova apareça em até 5 s (RNF15).
- Limite de requisições: com 2 ou mais servidores de aplicação, o contador precisa ser compartilhado. Se cada servidor contasse sozinho, o usuário teria o dobro do limite.
- O app continua funcionando se o Redis cair: o servidor de aplicação vai direto ao banco (cache-aside). O cache só acelera, não guarda dado que exista apenas nele.

2.7 Controle de concorrência na escrita

Escolha: transações curtas com bloqueio de linha onde há disputa.

Situação	Técnica
Várias contratações para a mesma vaga	SELECT ... FOR UPDATE na vaga e conferência de quantidade_profissionais antes de contratar
Vários freelancers aceitando a mesma substituição	UNIQUE em substituicao.nova_contratacao_id (já existe): o primeiro grava, os outros recebem "vaga já preenchida"
Candidatura duplicada	UNIQUE (vaga_id, freelancer_id) (já existe)
Crédito e saque na carteira	SELECT ... FOR UPDATE na carteira, e carteira + movimentacao_carteira na mesma transação
Webhook repetido do gateway	UNIQUE em chave_idempotencia e id_transacao_gateway (já existem)

Justificativa: o PostgreSQL garante essas regras dentro da transação (RNF20, integridade). Fazer a conferência só na aplicação deixaria passar duas requisições que chegam no mesmo milissegundo em servidores diferentes.

2.8 Pool de conexões

Escolha: PgBouncer em modo transação entre os servidores de aplicação/worker e o PostgreSQL.

Justificativa: cada conexão do PostgreSQL é um processo no servidor. Com 2 ou mais servidores de aplicação, workers e a réplica, o número de conexões abertas cresce rápido no pico. O pool reaproveita poucas conexões reais para muitas requisições curtas, que é o perfil da API REST stateless.

3. TÉCNICAS POR FLUXO CRÍTICO

Fluxo	Índices	Fila	Lote	Partição	Réplica	Cache	Bloqueio
Notificações em massa	Sim	Sim	Sim	Sim	Não	Não	Não
Busca de vagas	Sim	Não	Não	Não	Sim	Sim	Não
Candidatura/contratação	Sim	Sim (avisos e auditoria)	Não	Não (log_auditoria sim)	Só listas	Não	Sim
Chat	Sim	Sim (push)	Não	Não	Histórico antigo	Não	Não
Pagamento	Sim	Sim (webhook e liberação)	Não	Não	Não	Não	Sim

Resumo por fluxo:

- Notificações em massa: fila no PostgreSQL + INSERT em lote no worker + push em lotes de 500 + índice parcial de não lidas + partição mensal com retenção de 90 dias.
- Busca de vagas: índice parcial de vagas publicadas + réplica de leitura + cache da primeira página e do Serviço de Mapas.
- Candidatura e contratação: bloqueio da vaga ao contratar + UNIQUE já existentes + avisos e auditoria pela fila, fora da resposta ao usuário.
- Chat: push pelo Serviço de Notificações em vez de polling (o app só busca mensagens quando recebe o aviso ou abre a conversa) + índice por conversa + histórico paginado.
- Pagamento: tudo no banco principal, sem cache e sem réplica + idempotência + bloqueio da carteira + webhook e liberação pela fila.

FREELAS

4. COERÊNCIA COM O POSTGRESQL 16

Técnica	**Recurso usado**	**Disponível no PostgreSQL 16**
Índices compostos e parciais	CREATE INDEX ... WHERE	Sim, nativo
Fila de tarefas	FOR UPDATE SKIP LOCKED, LISTEN/NOTIFY	Sim, nativo
Escrita em lote	INSERT ... SELECT	Sim, nativo
Particionamento	PARTITION BY RANGE (data de criação)	Sim, nativo
Réplica de leitura	Replicação em streaming	Sim, nativo
Bloqueio e unicidade	SELECT ... FOR UPDATE, UNIQUE	Sim, nativo
Pool de conexões	PgBouncer	Ferramenta externa padrão do PostgreSQL
Cache	Redis	Peça separada, não guarda dado oficial

FREELAS